

ELEKTRA S.R.L.

Plataforma de Datos — Documentacion del Proyecto

Santa Fe, Argentina - April 2026

VERSION COMPLETA - Fases 1 a 5 finalizadas

Contenido

1. Que es este proyecto y para que sirve
2. Las herramientas que usamos
3. Estructura de carpetas y archivos
4. El archivo Excel - que habia y como estaba
5. FASE 1 - Limpieza y normalizacion de datos
 - 5.1 Script 01 - INSUMOS
 - 5.2 Script 02 - EQUIPOS
 - 5.3 Script 03 - CLIENTES
 - 5.4 Script 04 - PROVEEDORES
 - 5.5 Script 05 - PRODUCTOS y CATEGORIAS
 - 5.6 Script 06 - COSTOS DE PRODUCCION
6. FASE 2 - Esquema SQL y base de datos relacional
 - 6.1 Que es una base de datos relacional
 - 6.2 Las tablas creadas
 - 6.3 Las vistas de negocio
7. FASE 3 - Funciones de negocio en SQL
 - 7.1 Vistas analiticas
 - 7.2 Funciones de consulta
 - 7.3 Procedimientos de actualizacion de precios
8. FASE 4 - Dashboard con Streamlit
 - 8.1 Como levantar el dashboard
 - 8.2 Pagina Inicio
 - 8.3 Pagina Precios
 - 8.4 Pagina Rentabilidad
 - 8.5 Pagina Costos
9. FASE 5 - Automatizaciones
 - 9.1 Exportador de listas a Excel formateado
 - 9.2 Generador de facturas A/B en PDF
 - 9.3 API REST de precios con FastAPI
10. Resumen de resultados
11. Como usar el sistema dia a dia

1. Que es este proyecto y para que sirve

Elektra S.R.L. acumulo mas de 20 anos de informacion operativa dentro de un unico archivo de Excel con 32 hojas. La solucion que construimos transforma todo ese Excel en una plataforma de datos moderna con cinco componentes:

- **Base de datos:** Una base de datos PostgreSQL con todos los datos limpios y relacionados entre si.
- **Logica de negocio:** Funciones SQL que calculan costos, margenes y simulan aumentos de precios.
- **Dashboard:** Un dashboard web con graficos interactivos accesible desde cualquier navegador.
- **Excel automatico:** Un exportador que genera listas de precios en Excel con formato profesional.
- **Facturas y API:** Un generador de facturas A y B en PDF y una API REST para consultar precios.

Analogia simple: si el Excel era como tener todos los documentos de la empresa en una pila de papeles, la plataforma es como tener un sistema de gestion propio con busqueda instantanea, reportes automaticos y acceso desde cualquier dispositivo.

2. Las herramientas que usamos

Python 3.14

El lenguaje de programación con el que escribimos todos los scripts. Es el más usado en el mundo para análisis de datos y automatización.

pandas / openpyxl / xlrd

Librerías para leer el Excel, limpiar columnas, eliminar filas vacías y reorganizar información.

SQLAlchemy

El puente entre Python y la base de datos PostgreSQL.

PostgreSQL 18

La base de datos donde guardamos toda la información limpia. Gratuita, robusta, usada por empresas de todo el mundo.

pgAdmin 4

Interfaz visual para explorar y consultar la base de datos.

Streamlit

Framework Python para crear aplicaciones web interactivas sin necesidad de saber HTML o JavaScript.

Plotly

Librería de gráficos interactivos usada dentro del dashboard.

ReportLab

Librería para generar archivos PDF desde Python (facturas).

FastAPI + Uvicorn

Framework para crear APIs REST. Permite consultar precios desde cualquier aplicación o celular.

python-dotenv

Guarda la contraseña de la base de datos en un archivo separado (.env) que nunca se sube a internet.

3. Estructura de carpetas y archivos

```
ELEKTRA-plataforma-de-datos/
|
+-- ELEKTRA 2026.xls          <- Excel original (fuente)
+-- .env                     <- Credenciales DB (nunca subir)
+-- .gitignore               <- Ignora .env en Git
|
+-- scripts/
|   +-- 01_limpieza_insumos.py
|   +-- 02_limpieza_equipos.py
|   +-- 03_limpieza_clientes.py
|   +-- 04_limpieza_proveedores.py
|   +-- 05_limpieza_productos.py
|   +-- 06_limpieza_costos.py
|   +-- 07_crear_esquema_sql.py
|   +-- 08_funciones_negocio.py
|   +-- 09_exportar_listas_excel.py
|   +-- 10_generar_factura_pdf.py
|   +-- 11_api_precios.py
|   +-- generar_documentacion.py
|
+-- app/                      <- Dashboard Streamlit
|   +-- Inicio.py
|   +-- pages/
|       +-- 1_Precios.py
|       +-- 2_Rentabilidad.py
|       +-- 3_Clientes.py
|       +-- 4_Costos.py
|   +-- utils/
|       +-- db.py
|
+-- data/                     <- CSVs limpios + Excel exportado
+-- facturas/                 <- PDFs generados
```

4. El archivo Excel - que habia y como estaba

El archivo ELEKTRA 2026.xls tenia 32 hojas. Procesamos las hojas estructuradas:

Hoja	Contenido	Filas
INSUMOS	Materiales: codigo, descripcion, precio ARS/USD, proveedor, rubro, fecha	3.358
EQUIPOS	Productos terminados con precios publico y distribuidor en ARS y USD	2.132
CLIENTES	Clientes con CUIT, provincia, tipo IVA (172 columnas en total!)	1.763
PROVEEDORES	Proveedores con telefono, rubro, email, cotizacion dolar	259
PRODUCTOS	Catalogo con precios, IVA y categorias	2.131
CATEGORIAS	Tabla maestra de familias de productos	19
COSTOS	Costos de produccion por equipo (estructura en bloques)	12.207

Principales problemas encontrados en el Excel original:

- **Encoding roto:** Nombres de columnas con acentos, puntos y espacios que Python no puede manejar directamente.
- **Columnas vacias:** CLIENTES tenia 172 columnas, la mayoría sin nombre ni datos utiles.
- **Tipos de datos mixtos:** Columnas numericas con texto, fechas guardadas como texto.
- **Filas vacias:** El Excel usaba filas en blanco como separadores visuales.
- **Estructura compleja:** COSTOS no era una tabla plana sino bloques: nombre de equipo, lista de materiales, siguiente equipo...

5. FASE 1 - Limpieza y normalizacion de datos

La limpieza de datos es el paso mas importante. Cada script de limpieza sigue el mismo proceso: leer la hoja del Excel, renombrar columnas a snake_case, eliminar filas vacias, convertir tipos de datos, normalizar texto, eliminar duplicados, guardar CSV en /data y cargar a PostgreSQL.

Script	Hoja	Filas limpias	Principales ajustes
01_limpieza_insumos.py	INSUMOS	3.347	Columna extra con numero como nombre eliminada. Encoding de "Dolar" corregido.
02_limpieza_equipos.py	EQUIPOS	2.131	Columna ARTICULO duplicada eliminada. 2 columnas Unnamed eliminadas.
03_limpieza_clientes.py	CLIENTES	1.394	161 columnas vacias descartadas. Telefono armado de area + numero.
04_limpieza_proveedores.py	PROVEEDORES	254	Encoding de "dolar" corregido. Fechas de formato mixto normalizadas.
05_limpieza_productos.py	PRODUCTOS	2.131	Encoding de "Dolares" e "IdCategoria" corregido.
05_limpieza_productos.py	CATEGORIAS	19	Tabla maestra cargada primero como referencia.
06_limpieza_costos.py	COSTOS	5.752	Bloques de equipo parseados con algoritmo fila por fila. 2.072 equipos distintos.

5.3 CLIENTES - el caso mas complejo

La hoja de CLIENTES merece mencion especial. Tenia 172 columnas, la gran mayoria sin nombre ni datos. Identificamos las columnas utiles por posicion: col 1=codigo, 2=nombre, 3=fantasia, 4=contacto, 5=direccion, 6=codigo postal, 7=ciudad, 9-10=telefono, 30-31=telefono alternativo, 39=CUIT, 170=tipo IVA, 171=provincia. Descartamos las 161 columnas restantes.

5.6 COSTOS - estructura en bloques

Los costos estaban organizados en bloques: una fila con el nombre del equipo, luego filas de materiales, luego el siguiente equipo. Escribimos un algoritmo que recorre fila por fila y detecta cuando empieza un nuevo equipo (fila con nombre pero sin código de material) para transformar los bloques en una tabla plana.

6. FASE 2 - Esquema SQL y base de datos relacional

6.1 Que es una base de datos relacional

Una base de datos relacional conecta las tablas entre si a traves de claves. Por ejemplo, en el Excel si un insumo tiene "proveedor_codigo = 22", hay que ir a buscar manualmente en la hoja de proveedores que empresa es. En la base relacional esa conexion es automatica y permite hacer consultas que cruzan informacion de multiples tablas en milisegundos.

6.2 Las tablas creadas (script 07)

Tabla	Que contiene	Relacion
categorias	19 familias de productos	Tabla maestra
tipos_iva	Tipos de IVA posibles	Tabla maestra
provincias	Provincias argentinas	Tabla maestra
proveedores	254 proveedores con todos sus datos	Independiente
clientes	1.394 clientes con CUIT y tipo IVA	Independiente
insumos	3.347 materiales con precios ARS y USD	FK -> proveedores
productos	2.131 productos del catalogo de venta	FK -> categorias
equipos	2.131 equipos fabricados	Independiente
costos_equipos	5.752 registros material por equipo	Por nombre de equipo

6.3 Las vistas del script 07

Las vistas son consultas guardadas que se actualizan solas con los datos mas recientes:

- **v_costo_por_equipo:** Costo total de produccion de cada equipo sumando todos sus materiales.
- **v_margen_equipos:** Margen real, precio publico y precio distribuidor por equipo.
- **v_insumos_sin_stock:** Lista de insumos con stock = 0 o sin dato.
- **v_clientes_por_provincia:** Clientes agrupados por provincia y tipo de IVA.

7. FASE 3 - Funciones de negocio en SQL (script 08)

La Fase 3 agrega logica de negocio directamente en la base de datos: vistas analiticas mas avanzadas, funciones de consulta y procedimientos que modifican precios. Todo vive en el script 08_funciones_negocio.py.

7.1 Vistas analiticas nuevas

Vista	Para que sirve
v_rentabilidad_por_categoria	Precio promedio, precio USD y margen al distribuidor por cada familia de productos.
v_lista_precios_publica	Lista de precios al publico lista para exportar: sin IVA, con IVA, en USD.
v_lista_precios_distribuidor	Lista de precios para distribuidores con descuento.
v_materiales_mas_usados	Materiales que aparecen en mas equipos distintos con impacto en costos totales.
v_equipos_rentabilidad	Margen real, margen al distribuidor y precio USD por cada equipo fabricado.
v_insumos_caros_sin_usd	Insumos sin precio en dolares, con estimacion calculada desde ARS / cotizacion.

7.2 Funciones de consulta

Las funciones se llaman con SELECT y devuelven datos sin modificar nada:

```
-- Ver el desglose completo de materiales del equipo VIBRO:
SELECT * FROM fn_detalle_costo_equipo('VIBRO');

-- Ver el costo total de produccion del RADAR:
SELECT fn_costo_total_equipo('RADAR');

-- Convertir 100.000 pesos a dolares con cotizacion 1.500:
SELECT fn_ars_a_usd(100000, 1500);

-- Simular como quedarian los precios con un aumento del 20%:
SELECT * FROM fn_simular_aumento_insumos(20);
SELECT * FROM fn_simular_aumento_equipos(20);
```

7.3 Procedimientos de actualizacion de precios

Los procedimientos modifican los datos de la base. Se llaman con CALL y tienen una validacion que impide porcentajes invalidos (menores a 0 o mayores a 500):

```
-- Aplicar 15.5%% de aumento solo a insumos:
CALL sp_aumento_insumos(15.5);

-- Aplicar 15.5%% de aumento solo a equipos:
CALL sp_aumento_equipos(15.5);

-- Aplicar 15.5%% de aumento solo a productos:
```

```
CALL sp_aumento_productos(15.5);  
  
-- Aplicar 15.5%% a todo de una vez (insumos + equipos + productos):  
CALL sp_aumento_general(15.5);
```

IMPORTANTE: Siempre usar primero `fn_simular_aumento_insumos()` para ver como quedarían los precios ANTES de llamar al procedimiento que los modifica. Una vez aplicado el aumento, la base queda actualizada.

8. FASE 4 - Dashboard con Streamlit

8.1 Como levantar el dashboard

Desde la terminal, pararse en la carpeta app/ y ejecutar:

```
py -m streamlit run Inicio.py
```

Se abre automaticamente en el navegador en <http://localhost:8501>

8.2 Pagina Inicio

La pagina principal muestra un resumen ejecutivo del negocio: 5 contadores (insumos, equipos, clientes, proveedores, equipos con costo calculado), top 5 categorias por precio promedio, top 5 equipos por margen real, grafico de clientes por provincia y materiales mas usados en produccion.

8.3 Pagina Precios

Lista de precios interactiva con filtros por categoria y buscador por nombre. Permite elegir entre lista publica y lista distribuidor. Tiene un control deslizante para ajustar la cotizacion del dolar y ver al instante cuanto vale cada producto en USD. Incluye boton para descargar la lista filtrada como archivo Excel.

8.4 Pagina Rentabilidad

Tres pestanas de analisis:

- **Por categoria:** Grafico de barras de precio promedio por categoria y margen al distribuidor por categoria.
- **Por equipo:** Grafico scatter (nube de puntos) de costo vs precio con linea de break-even. Un punto encima de la linea roja significa ganancia; debajo significa perdida.
- **Desglose de costo:** Selector de equipo especifico que muestra el grafico de torta con la composicion del costo (que porcentaje representa cada material) y la tabla de materiales completa.

8.5 Pagina Costos

Tres pestanas:

- **Insumos:** Buscador de insumos por descripcion y rubro, con grafico de precio promedio por rubro.
- **Equipos:** Ranking de equipos por costo de produccion total con slider para elegir cuantos mostrar, y desglose detallado del equipo seleccionado.
- **Actualizar precios:** Simulador (muestra como quedarian los precios sin modificarlos) y aplicador de aumentos con checkbox de confirmacion para evitar errores accidentales.

9. FASE 5 - Automatizaciones

9.1 Exportador de listas a Excel formateado (script 09)

El script 09_exportar_listas_excel.py genera un archivo Excel profesional con tres hojas, cada una con su propio esquema de colores, filas alternadas, separadores por categoria o rubro, columnas con ancho automatico y encabezados fijos (freeze panes):

Hoja	Color	Contenido
Lista Publica	Azul	1.980 productos con precio sin IVA, con IVA, en USD y descuento distribuidor
Lista Distribuidor	Verde	1.979 productos con precio distribuidor sin y con IVA
Insumos	Naranja	3.342 materiales con precio ARS, ARS+IVA, USD, cotizacion y stock

Para ejecutarlo:

```
py scripts/09_exportar_listas_excel.py
```

El archivo se guarda en data/ELEKTRA_Listas_Precios_AAAAMMDD.xlsx con la fecha del dia.

9.2 Generador de facturas A/B en PDF (script 10)

El script 10_generar_factura_pdf.py genera una factura en PDF lista para imprimir. El tipo de factura (A o B) se determina automaticamente segun el tipo de IVA del cliente:

- **Factura A:** Factura A: para clientes "Responsable Inscripto". Muestra IVA discriminado.
- **Factura B:** Factura B: para "Consumidor Final", "Monotributista" y "Exento". IVA incluido en el precio.

Contenido de la factura generada:

- Encabezado con datos de Elektra SRL: nombre, direccion, CUIT, inicio de actividades.
- Recuadro con la letra A o B bien visible (requerimiento AFIP).
- Datos del cliente: razon social, direccion, ciudad, CUIT, tipo de IVA.
- Tabla de items: descripcion, cantidad, precio unitario, subtotal y (si es tipo A) IVA.
- Totales: subtotal neto, IVA y total a pagar.

Formas de usar el script:

```
# Generar factura de demostracion (toma el primer cliente con CUIT):
py scripts/10_generar_factura_pdf.py --demo

# Generar factura para el cliente 15 con 1 RADAR y 2 VIBROs:
py scripts/10_generar_factura_pdf.py --cliente 15 --items
```

```
"RADAR:1,VIBRO:2"

# Con numero de factura personalizado:
py scripts/10_generar_factura_pdf.py --cliente 15 --items "RADAR:1" --
numero "0001-00000042"
```

Los PDFs se guardan en la carpeta facturas/ con el nombre: FACTURA_A_NUMERO_CLIENTE.pdf

9.3 API REST de precios con FastAPI (script 11)

El script 11_api_precios.py crea una API REST que permite consultar precios, insumos y costos desde cualquier aplicacion, celular o sistema externo mediante peticiones HTTP simples.

Como levantarla:

```
py -m uvicorn scripts/11_api_precios:app --reload --port 8000
```

Una vez corriendo, FastAPI genera automaticamente una documentacion interactiva en <http://localhost:8000/docs> donde se puede probar cada endpoint con un formulario visual.

Endpoints disponibles:

Endpoint	Metodo	Que devuelve
/productos	GET	Lista de precios al publico con filtros por categoria y nombre
/productos/{nombre}	GET	Precio publico Y distribuidor de un producto especifico
/insumos	GET	Lista de insumos con filtros por rubro y descripcion
/insumos/{codigo}	GET	Detalle completo de un insumo por codigo
/equipos	GET	Lista de equipos con margen real y precios
/equipos/{nombre}/costo	GET	Costo de produccion completo de un equipo con desglose
/clientes	GET	Lista de clientes con filtros por provincia y tipo IVA
/clientes/{codigo}	GET	Datos completos de un cliente
/categorias	GET	Categorias con precio promedio y margen
/cotizacion	GET	Cotizaciones de dolar registradas por proveedor
/stock-critico	GET	Insumos con stock = 0 o sin dato

Ejemplo de consulta desde el navegador o cualquier programa:

```
http://localhost:8000/productos/RADAR
-> Devuelve: precio sin IVA, con IVA, en USD, precio distribuidor

http://localhost:8000/equipos/VIBRO/costo
-> Devuelve: costo total y lista de todos los materiales con
cantidades

http://localhost:8000/clientes?provincia=SANTA FE&tipo_iva=responsable
-> Devuelve: lista de clientes RI de Santa Fe
```

10. Resumen de resultados

Al completar las 5 fases, la plataforma contiene:

Componente	Detalle	Estado
Base de datos PostgreSQL	elektra_srl con 9 tablas y datos limpios	ACTIVO
Insumos cargados	3.347 materiales con precios ARS y USD	ACTIVO
Equipos cargados	2.131 equipos con precios y margenes	ACTIVO
Clientes cargados	1.394 clientes con CUIT y tipo IVA	ACTIVO
Proveedores cargados	254 proveedores con cotizacion dolar	ACTIVO
Costos de produccion	5.752 registros - 2.072 equipos distintos	ACTIVO
Vistas SQL	10 vistas analiticas de negocio	ACTIVO
Funciones SQL	7 funciones de consulta y conversion	ACTIVO
Procedimientos SQL	4 procedimientos de actualizacion de precios	ACTIVO
Dashboard Streamlit	4 paginas: Inicio, Precios, Rentabilidad, Costos	ACTIVO
Exportador Excel	Genera lista publica + distribuidor + insumos formateados	ACTIVO
Generador PDF	Facturas A y B con datos del cliente y detalle de items	ACTIVO
API REST	11 endpoints documentados con FastAPI	ACTIVO
Documentacion	Este documento Word generado automaticamente	ACTIVO

11. Como usar el sistema dia a dia

Consultar precios

Opcion 1 - Dashboard (la mas comoda):

```
cd app
py -m streamlit run Inicio.py
```

Opcion 2 - API desde el navegador:

```
py -m uvicorn scripts/11_api_precios:app --port 8000
# Abrir: http://localhost:8000/productos/RADAR
```

Aplicar un aumento de precios

Desde pgAdmin (recomendado para mayor control):

```
-- Primero SIMULAR (no modifica nada):
SELECT * FROM fn_simular_aumento_insumos(15);

-- Si el resultado es correcto, APLICAR:
CALL sp_aumento_general(15);
```

Desde el dashboard (mas sencillo):

- Ir a la pagina "Costos".
- Abrir la pestana "Actualizar precios".
- Ingresar el porcentaje en "Simular" y verificar los precios nuevos.
- Si esta correcto, ingresar el mismo porcentaje en "Aplicar", tildar la confirmacion y presionar el boton.

Generar una lista de precios en Excel

```
py scripts/09_exportar_listas_excel.py
```

El archivo queda en data/ELEKTRA_Listas_Precios_AAAAMMDD.xlsx

Generar una factura PDF

```
# Reemplazar 15 por el codigo del cliente y los productos reales:
py scripts/10_generar_factura_pdf.py --cliente 15 --items
"RADAR:1,VIBRO:2"
```

La factura queda en la carpeta facturas/

Actualizar los datos cuando cambia el Excel

Si Elektra actualiza el archivo Excel fuente, ejecutar los scripts de limpieza en orden:

```
cd scripts
py 01_limpieza_insumos.py
py 02_limpieza_equipos.py
```

```
py 03_limpieza_clientes.py  
py 04_limpieza_proveedores.py  
py 05_limpieza_productos.py  
py 06_limpieza_costos.py
```

Los scripts usan `if_exists="replace"` lo que significa que borran la tabla vieja y cargan los datos nuevos. El dashboard y la API reflejan los cambios automaticamente.